```
def update(self):
    for item in self.scheduled[:]:
        item["delay"] -= 1
        if item["delay"] <= 0:
            self.play(item["name"], item["volume"], item["loops"], item["channel"])
            self.scheduled.remove(item)
    if self._duck_frames > 0:
        self._duck_frames -= 1
        if self._duck_frames <= 0:
            self._duck_factor = 1.0
        for name, ch in list(self._loop_channels.items()):
            if ch is None:
                continue
            base = self._loop_base_volumes.get(name, 1.0) if hasattr(self, '_loop_base_volumes') else self._loop_base_volumes.get(name, 1.0)
            try:
                ch.set_volume(self._effective_volume(name, base))
            except Exception:
                pass

def duck_ambience(self, frames=60, factor=0.35):
    self._duck_frames = max(self._duck_frames, int(frames))
    self._duck_factor = min(self._duck_factor, max(0.0, min(1.0, factor)))

def play_3d(self, name, source_pos, listener_pos, max_dist=800, base_volume=1.0):
    s = self.sounds.get(name)
    if s is None:
        return None
    dx = source_pos[0] - listener_pos[0]
    dy = source_pos[1] - listener_pos[1]
    dist = math.hypot(dx, dy)
    if dist >= max_dist:
        return None
    attenuation = max(0.0, 1.0 - (dist / max_dist))
    final_vol = base_volume * self.master_volume * self.sfx_volume * attenuation
    pan = dx / max_dist
    pan = max(-1.0, min(1.0, pan))
    left_vol = final_vol * min(1.0, 1.0 - pan)
    right_vol = final_vol * min(1.0, 1.0 + pan)
    channel = s.play()
    if channel:
        channel.set_volume(left_vol, right_vol)
    return channel

def stop(self, name=None, channel=None):
    if channel is not None and channel in _channels:
        _channels[channel].stop()
    elif name and name in self.sounds:
        self.sounds[name].stop()
        self._loop_channels.pop(name, None)
        self._loop_base_volumes.pop(name, None)

def stop_all(self):
    for ch in _channels.values():
        ch.stop()
    for name in self.sounds:
        try:
            self.sounds[name].stop()
        except Exception:
            pass
    self._loop_channels.clear()
    self._loop_base_volumes.clear()
    self.scheduled.clear()

def set_master_volume(self, vol_0_to_100):
    self.master_volume = max(0.0, min(1.0, vol_0_to_100 / 100.0))
    try:
        pygame.mixer.music.set_volume(self.master_volume * getattr(self, "_current_music_vol", 0.5))
    except Exception:
        pass

def play_music(self, filename, volume=0.5):
    """Loads and streams an OGG music track in the background."""
    import os
    filepath = os.path.join(os.path.dirname(__file__), filename)
    if not os.path.exists(filepath):
        filepath = filename
    if not os.path.exists(filepath):
        print(f"Music file not found: {filename}")
        return False
    try:
        pygame.mixer.music.load(filepath)
        self._current_music_vol = volume
        pygame.mixer.music.set_volume(self.master_volume * volume)
        pygame.mixer.music.play(loops=-1)
        return True
    except Exception as e:
        print(f"Failed to play music: {e}")
        return False

def stop_music(self):
    """Stops the streaming background music."""
    try:
        pygame.mixer.music.stop()
    except Exception:
        pass

def set_music_volume(self, volume):
    """Adjusts the streaming music volume dynamically."""
    self._current_music_vol = volume
    try:
        pygame.mixer.music.set_volume(self.master_volume * volume)
    except Exception:
        pass

def set_sfx_volume(self, volume):
    """Adjusts the SFX volume scale dynamically.

    Affects all one-shot sound effects + looping ambience. Also refreshes
    any currently-looping sounds so the change is audible immediately
    rather than only on the next play().
    """
    self.sfx_volume = max(0.0, min(1.0, float(volume)))
    for name, ch in list(self._loop_channels.items()):
        if ch is None:
            continue
        base = self._loop_base_volumes.get(name, 1.0)
        try:
            ch.set_volume(self._effective_volume(name, base))
        except Exception:
            pass

def mute_all(self):
    """Pause every playing sound + streaming music. Used by the
    Audio → Mute on Focus Loss setting when the window loses focus."""
    try:
        pygame.mixer.pause()
        pygame.mixer.music.pause()
    except Exception:
        pass

def unmute_all(self):
    """Resume everything paused by mute_all()."""
    try:
        pygame.mixer.unpause()
        pygame.mixer.music.unpause()
    except Exception:
        pass

def _load_optional_sound(self, name, candidate_paths):
    """Try to load an external sound file into self.sounds[name].

    Used for user-provided assets like assets/alarm.mp3. If loading fails,
    nothing crashes — gameplay can fall back to the procedural sound.
    """
    for path in candidate_paths:
        try:
            if path and os.path.exists(path):
                self.sounds[name] = pygame.mixer.Sound(path)
                print(f"Loaded optional sound '{name}': {path}")
                return True
        except Exception as e:
            print(f"Failed loading optional sound '{name}' from {path}: {e}")
    return False

def play_shutdown_alarm(self, volume=0.85):
    """Start Phase 3 lockdown alarm loop.

    The external alarm.mp3 is clearly declared loud, so this defaults low.
    Uses assets/alarm.mp3 if loaded as shutdown_alarm_loop.
    Falls back to the procedural alarm if the MP3 is unavailable.
    """
    if "shutdown_alarm_loop" in self.sounds:
        return self.play("shutdown_alarm_loop", volume=volume, loops=-1, channel="p3_alarm")

    # Fallback procedural alarm, also quieter.
    return self.play("alarm", volume=min(1.0, volume * 0.65), loops=-1, channel="p3_alarm")

def stop_shutdown_alarm(self):
    """Stop Phase 3 lockdown alarm loop."""
    try:
        self.stop("shutdown_alarm_loop")
    except Exception:
        pass
    try:
        self.stop("alarm")
    except Exception:
        pass
    try:
        self.stop(channel="p3_alarm")
    except Exception:
        pass

def get_sound(self, name):
    return self.sounds.get(name)

def is_playing(self, channel):
    if channel in _channels:
        return _channels[channel].get_busy()
    return False

def fadeout(self, channel, fade_ms=500):
    if channel in _channels:
        _channels[channel].fadeout(fade_ms)
```